02 Data Collection and Ingestion

Course Name: Data Management for Machine Learning

Assignment Title: End-to-End Data Management Pipeline for a Recommendation System

Assignment: Group 51 - Data management for Machine Learning Group 51

Team Members

Team Member Name	Team Member ID
BANSHIDHAR RATH	2025AE05346
JITENDRA KUMAR TIWARI	2025AE05518
KATBA ANKIT CHIMANBHAI	2025AE05229
NAVEEN SURATHU	2025AE05492

1. Objective of Data Collection and Ingestion

This section documents how the recommendation pipeline ingests data from multiple source types, stores collected data in structured raw and bronze layers, and preserves logs and reports for monitoring and auditability. The evidence below is automatically collected from the current project folder so that the PDF reflects the actual implementation state.

2. Ingestion Summary

The project collects at least two categories of source data: behavioral and catalog CSV files, and external API-style JSON product data. The notebook scans the project folders and summarizes the discovered ingestion assets below.

Source Type	File Name	Relative Path	Key Attributes / Columns	Last Modified
RetailRocket CSV	category_tree.csv	data\raw\retailrocket\load_date=2026-04-29\load_hour=11\category_tree.csv	categoryid, parentid	2026-04-20 17:27:59
RetailRocket CSV	events.csv	data\raw\retailrocket\load_date=2026-04-29\load_hour=11\events.csv	timestamp, visitorid, event, itemid, transactionid	2026-04-20 17:28:00
RetailRocket CSV	item_properties_part1.csv	data\raw\retailrocket\load_date=2026-04-29\load_hour=11\item_properties_part1.csv	timestamp, itemid, property, value	2026-04-20 17:28:03
RetailRocket CSV	item_properties_part2.csv	data\raw\retailrocket\load_date=2026-04-29\load_hour=11\item_properties_part2.csv	timestamp, itemid, property, value	2026-04-20 17:28:05
DummyJSON Raw JSON	categories_raw.json	data\raw\dummyjson\load_date=2026-04-29\load_hour=11\categories_raw.json	slug, name, url	2026-04-29 16:42:35
DummyJSON Raw JSON	products_raw.json	data\raw\dummyjson\load_date=2026-04-29\load_hour=11\products_raw.json	id, title, description, category, price, discountPercentage, rating, stock	2026-04-29 16:42:35
Bronze Parquet	categories.parquet	data\bronze\dummyjson\categories.parquet	slug, name, url, source_system, source_file, source_mode, batch_id, ingestion_ts	2026-04-29 16:42:35
Bronze Parquet	products.parquet	data\bronze\dummyjson\products.parquet	id, title, description, category, price, discountPercentage, rating, stock	2026-04-29 16:42:35
Bronze Parquet	category_tree.parquet	data\bronze\retailrocket\category_tree.parquet	categoryid, parentid, source_system, source_file, batch_id, ingestion_ts	2026-04-29 16:42:58

Source Type	File Name	Relative Path	Key Attributes / Columns	Last Modified
Bronze Parquet	events.parquet	data\bronze\retailrocket\events.parquet	timestamp, visitorid, event, itemid, transactionid, source_system, source_file, batch_id	2026-04-29 16:42:58
Bronze Parquet	item_properties.parquet	data\bronze\retailrocket\item_properties.parquet	timestamp, itemid, property, value, source_system, source_file, batch_id, ingestion_ts	2026-04-29 16:43:30

3. Data Sources and Attributes

- RetailRocket CSV files capture user interactions and item metadata such as events, categories, timestamps, and item properties.
- DummyJSON raw JSON files provide product and category information used to enrich item-level context.
- Bronze parquet datasets represent ingested and standardized downstream storage for curated use.
- Together, these sources support recommendation use cases such as interaction modeling, item enrichment, and downstream validation.

4. Ingestion and Validation Notebooks Used

The following notebooks were identified from the project as relevant to ingestion or validation activities. Their notebook names, locations, saved code cells, and captured outputs are shown below.

Notebook Name	Relative Path	Last Modified	Size
dummyjson_ingestion_executed.ipynb	runs\prefect\20260429_115432\executed_notebooks\ingest_dummyjson\dummyjson_ingestion_executed.ipynb	2026-04-29 11:56:07	19.97 KB
retailrocket_ingestion_executed.ipynb	runs\prefect\20260429_115432\executed_notebooks\ingest_retailrocket\retailrocket_ingestion_executed.ipynb	2026-04-29 11:58:25	19.74 KB
run_validation_executed.ipynb	runs\prefect\20260429_115432\executed_notebooks\validate_raw\run_validation_executed.ipynb	2026-04-29 12:02:20	49.57 KB
dummyjson_ingestion_executed.ipynb	runs\prefect\20260429_121758\executed_notebooks\ingest_dummyjson\dummyjson_ingestion_executed.ipynb	2026-04-29 12:18:31	19.97 KB
retailrocket_ingestion_executed.ipynb	runs\prefect\20260429_121758\executed_notebooks\ingest_retailrocket\retailrocket_ingestion_executed.ipynb	2026-04-29 12:19:50	19.74 KB
run_validation_executed.ipynb	runs\prefect\20260429_121758\executed_notebooks\validate_raw\run_validation_executed.ipynb	2026-04-29 12:23:01	49.57 KB
dummyjson_ingestion_executed.ipynb	runs\prefect\20260429_132217\executed_notebooks\ingest_dummyjson\dummyjson_ingestion_executed.ipynb	2026-04-29 13:23:10	19.97 KB
retailrocket_ingestion_executed.ipynb	runs\prefect\20260429_132217\executed_notebooks\ingest_retailrocket\retailrocket_ingestion_executed.ipynb	2026-04-29 13:24:17	19.74 KB
run_validation_executed.ipynb	runs\prefect\20260429_132217\executed_notebooks\validate_raw\run_validation_executed.ipynb	2026-04-29 13:27:08	49.57 KB

Notebook Name	Relative Path	Last Modified	Size
dummyjson_ingestion_executed.ipynb	runs\prefect\20260429_155743\executed_notebooks\ingest_dummyjson\dummyjson_ingestion_executed.ipynb	2026-04-29 15:59:10	19.59 KB

Notebook: dummyjson_ingestion_executed.ipynb

Path:
runs\prefect\20260429_115432\executed_notebooks\ingest_dummyjson\dummyjson_ingestion_executed.ipynb

Code Cell 1

```
# DM4ML -Assignment - Ingestion - API

import json
import time
import random
from pathlib import Path
from datetime import datetime, timezone

import pandas as pd
import requests

# ============================================================
# RecoMart - DummyJSON API Ingestion Script
# - Fetches product + category data from DummyJSON
# - Retries on transient failures
# - Falls back to embedded sample data if API is unreachable
# - Stores raw JSON, bronze parquet, logs, and manifest
# ============================================================

# -----------------------------
# 1) CONFIG
# -----------------------------
PROJECT_ROOT = Path.cwd().resolve()
while not (PROJECT_ROOT / "src").exists() and PROJECT_ROOT != PROJECT_ROOT.parent:
    PROJECT_ROOT = PROJECT_ROOT.parent
DATA_ROOT = PROJECT_ROOT / "data"
SOURCE_NAME = "dummyjson"

USE_OFFLINE_FALLBACK = True
MAX_ATTEMPTS = 5
REQUEST_TIMEOUT = 30
PAGE_SIZE = 30

UTC_NOW = datetime.now(timezone.utc)
LOAD_DATE = UTC_NOW.strftime("%Y-%m-%d")
LOAD_HOUR = UTC_NOW.strftime("%H")
BATCH_ID = UTC_NOW.strftime("%Y%m%dT%H%M%SZ")
INGESTION_TS = UTC_NOW.isoformat()

BASE_URL = "https://dummyjson.com"
PRODUCTS_ENDPOINT = f"{BASE_URL}/products"
CATEGORIES_ENDPOINT = f"{BASE_URL}/products/categories"

RAW_DIR = PROJECT_ROOT / "data" / "raw" / SOURCE_NAME / f"load_date={LOAD_DATE}" /
f"load_hour={LOAD_HOUR}"
BRONZE_DIR = PROJECT_ROOT / "data" / "bronze" / SOURCE_NAME
LOG_DIR = PROJECT_ROOT / "logs"
METADATA_DIR = PROJECT_ROOT / "metadata"

RAW_DIR.mkdir(parents=True, exist_ok=True)
BRONZE_DIR.mkdir(parents=True, exist_ok=True)
LOG_DIR.mkdir(parents=True, exist_ok=True)
METADATA_DIR.mkdir(parents=True, exist_ok=True)

LOG_FILE = LOG_DIR / f"{SOURCE_NAME}_ingestion_log.jsonl"
MANIFEST_FILE = METADATA_DIR / f"{SOURCE_NAME}_manifest_{BA
```

Output Cell 1

```
c:\Users\barath\AppData\Local\anaconda3\Lib\site-
packages\pandas\core\computation\expressions.py:22: UserWarning: Pandas requires version
'2.10.2' or newer of 'numexpr' (version '2.10.1' currently installed).
  from pandas.core.computation.check import NUMEXPR_INSTALLED


DummyJSON ingestion completed successfully
Source mode: api
Raw dir: C:\Users\barath\recomart-pipeline\data\raw\dummyjson\load_date=2026-04-29\load_hour=06
Bronze dir: C:\Users\barath\recomart-pipeline\data\bronze\dummyjson
Manifest: C:\Users\barath\recomart-pipeline\metadata\dummyjson_manifest_20260429T062555Z.json
Summary:
```

```
      dataset  rows  columns  \
0    products   194       32
1  categories    24        8

                                         output_path source_mode
0  C:\Users\barath\recomart-pipeline\data\bronze\...         api
1  C:\Users\barath\recomart-pipeline\data\bronze\...         api
```

Code Cell 2

```
No code found.
```

Output Cell 2

```
No saved output found in notebook.
```

Notebook: retailrocket_ingestion_executed.ipynb

Path:
runs\prefect\20260429_115432\executed_notebooks\ingest_retailrocket\retailrocket_ingestion_executed.ipynb

Code Cell 1

```
# DM4ML -Assignment - Ingestion - File

# ============================================================
# Retailrocket Ingestion Notebook / Script
# Purpose:
#   - Download Retailrocket dataset via kagglehub
#   - Copy raw files into partitioned local data lake folders
#   - Log success/failure events
#   - Create a manifest for audit/lineage
#   - Build bronze parquet datasets for downstream use
# ============================================================

# -----------------------------
# 0) Optional: install packages
# Uncomment if needed in Jupyter
# -----------------------------
# import sys
# !{sys.executable} -m pip install kagglehub pandas pyarrow

import json
import shutil
import time
from pathlib import Path
from datetime import datetime, timezone

import pandas as pd
import kagglehub

# =============================
# 1) CONFIG
# =============================
PROJECT_ROOT = Path.cwd().resolve()
while not (PROJECT_ROOT / "src").exists() and PROJECT_ROOT != PROJECT_ROOT.parent:
    PROJECT_ROOT = PROJECT_ROOT.parent
DATA_ROOT = PROJECT_ROOT / "data"
SOURCE_NAME = "retailrocket"
DATASET_ID = "retailrocket/ecommerce-dataset"

UTC_NOW = datetime.now(timezone.utc)
LOAD_DATE = UTC_NOW.strftime("%Y-%m-%d")
LOAD_HOUR = UTC_NOW.strftime("%H")
BATCH_ID = UTC_NOW.strftime("%Y%m%dT%H%M%SZ")
INGESTION_TS = UTC_NOW.isoformat()

RAW_DIR = PROJECT_ROOT / "data" / "raw" / SOURCE_NAME / f"load_date={LOAD_DATE}" /
f"load_hour={LOAD_HOUR}"
BRONZE_DIR = PROJECT_ROOT / "data" / "bronze" / SOURCE_NAME
LOG_DIR = PROJECT_ROOT / "logs"
METADATA_DIR = PROJECT_ROOT / "metadata"

RAW_DIR.mkdir(parents=True, exist_ok=True)
BRONZE_DIR.mkdir(parents=True, exist_ok=True)
LOG_DIR.mkdir(parents=True, exist_ok=True)
METADATA_DIR.mkdir(parents=True, exist_ok=True)

LOG_FILE = LOG_DIR / f"{SOURCE
```

Output Cell 1

```
c:\Users\barath\AppData\Local\anaconda3\Lib\site-
packages\pandas\core\computation\expressions.py:22: UserWarning: Pandas requires version
'2.10.2' or newer of 'numexpr' (version '2.10.1' currently installed).
  from pandas.core.computation.check import NUMEXPR_INSTALLED
Downloaded dataset to: C:\Users\barath\.cache\kagglehub\datasets\retailrocket\ecommerce-
```

```
dataset\versions\2


Raw files copied to: C:\Users\barath\recomart-
pipeline\data\raw\retailrocket\load_date=2026-04-29\load_hour=06
Manifest saved to: C:\Users\barath\recomart-
pipeline\metadata\retailrocket_manifest_20260429T062607Z.json
```

Code Cell 2

```
No code found.
```

Output Cell 2

```
No saved output found in notebook.
```

Notebook: run_validation_executed.ipynb

Path: runs\prefect\20260429_115432\executed_notebooks\validate_raw\run_validation_executed.ipynb

Code Cell 1

```
# DM4ML - Assignment - Validation + Auto-Fix + Revalidation + PDF Report

import json
from pathlib import Path
from datetime import datetime, timezone

import pandas as pd

from reportlab.lib import colors
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import getSampleStyleSheet
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle,
PageBreak

# ============================================================
# RecoMart Validation - robust file discovery + validation
# with remediation, revalidation, and PDF reporting
# ============================================================

PROJECT_ROOT = Path.cwd().resolve()
while not (PROJECT_ROOT / "src").exists() and PROJECT_ROOT != PROJECT_ROOT.parent:
    PROJECT_ROOT = PROJECT_ROOT.parent

RAW_ROOT = PROJECT_ROOT / "data" / "raw"
BRONZE_ROOT = PROJECT_ROOT / "data" / "bronze"

REMEDIATED_RAW_ROOT = PROJECT_ROOT / "data" / "remediated" / "raw"
REMEDIATED_BRONZE_ROOT = PROJECT_ROOT / "data" / "remediated" / "bronze"

REPORT_ROOT = PROJECT_ROOT / "reports" / "validation"
LOG_ROOT = PROJECT_ROOT / "logs"

for folder in [REPORT_ROOT, LOG_ROOT, REMEDIATED_RAW_ROOT, REMEDIATED_BRONZE_ROOT]:
    folder.mkdir(parents=True, exist_ok=True)

UTC_NOW = datetime.now(timezone.utc)
RUN_ID = UTC_NOW.strftime("%Y%m%dT%H%M%SZ")
RUN_TS = UTC_NOW.isoformat()

LOG_FILE = LOG_ROOT / f"validation_log_{RUN_ID}.jsonl"
INITIAL_ISSUES_FILE = REPORT_ROOT / f"validation_issues_initial_{RUN_ID}.csv"
INITIAL_SUMMARY_FILE = REPORT_ROOT / f"validation_summary_initial_{RUN_ID}.csv"
FINAL_ISSUES_FILE = REPORT_ROOT / f"validation_issues_revalidated_{RUN_ID}.csv"
FINAL_SUMMARY_FILE = REPORT_ROOT / f"validation_summary_revalidated_{RUN_ID}.csv"
FIX_LOG_FILE = REPORT_ROOT / f"fix_log_{RUN_ID}.csv"
REPORT_FILE = REPO
```

Output Cell 1

```
c:\Users\barath\AppData\Local\anaconda3\Lib\site-
packages\pandas\core\computation\expressions.py:22: UserWarning: Pandas requires version
'2.10.2' or newer of 'numexpr' (version '2.10.1' currently installed).
  from pandas.core.computation.check import NUMEXPR_INSTALLED


PROJECT_ROOT: C:\Users\barath\recomart-pipeline
RAW_ROOT: C:\Users\barath\recomart-pipeline\data\raw
BRONZE_ROOT: C:\Users\barath\recomart-pipeline\data\bronze

Discovered files:
- events_csv: C:\Users\barath\recomart-
pipeline\data\raw\retailrocket\load_date=2026-04-29\load_hour=06\events.csv
- category_tree_csv: C:\Users\barath\recomart-
pipeline\data\raw\retailrocket\load_date=2026-04-26\load_hour=19\category_tree.csv
- item_properties_part1_csv: C:\Users\barath\recomart-
pipeline\data\raw\retailrocket\load_date=2026-04-29\load_hour=06\item_properties_part1.csv
```

```
- item_properties_part2_csv: C:\Users\barath\recomart-
pipeline\data\raw\retailrocket\load_date=2026-04-29\load_hour=06\item_properties_part2.csv
- products_raw_json: C:\Users\barath\recomart-
pipeline\data\raw\dummyjson\load_date=2026-04-29\load_hour=06\products_raw.json
- categories_raw_json: C:\Users\barath\recomart-pipeline\data\raw\dummyjson\load_date=20
```

Code Cell 2

```
No code found.
```

Output Cell 2

```
No saved output found in notebook.
```

Notebook: dummyjson_ingestion_executed.ipynb

Path:

runs\prefect\20260429_121758\executed_notebooks\ingest_dummyjson\dummyjson_ingestion_executed.ipynb

Code Cell 1

```
# DM4ML -Assignment - Ingestion - API

import json
import time
import random
from pathlib import Path
from datetime import datetime, timezone

import pandas as pd
import requests

# ==============================================================
# RecoMart - DummyJSON API Ingestion Script
# - Fetches product + category data from DummyJSON
# - Retries on transient failures
# - Falls back to embedded sample data if API is unreachable
# - Stores raw JSON, bronze parquet, logs, and manifest
# ==============================================================

# -----------------------------
# 1) CONFIG
# -----------------------------
PROJECT_ROOT = Path.cwd().resolve()
while not (PROJECT_ROOT / "src").exists() and PROJECT_ROOT != PROJECT_ROOT.parent:
    PROJECT_ROOT = PROJECT_ROOT.parent
DATA_ROOT = PROJECT_ROOT / "data"
SOURCE_NAME = "dummyjson"

USE_OFFLINE_FALLBACK = True
MAX_ATTEMPTS = 5
REQUEST_TIMEOUT = 30
PAGE_SIZE = 30

UTC_NOW = datetime.now(timezone.utc)
LOAD_DATE = UTC_NOW.strftime("%Y-%m-%d")
LOAD_HOUR = UTC_NOW.strftime("%H")
BATCH_ID = UTC_NOW.strftime("%Y%m%dT%H%M%SZ")
INGESTION_TS = UTC_NOW.isoformat()

BASE_URL = "https://dummyjson.com"
PRODUCTS_ENDPOINT = f"{BASE_URL}/products"
CATEGORIES_ENDPOINT = f"{BASE_URL}/products/categories"

RAW_DIR = PROJECT_ROOT / "data" / "raw" / SOURCE_NAME / f"load_date={LOAD_DATE}" /
f"load_hour={LOAD_HOUR}"
BRONZE_DIR = PROJECT_ROOT / "data" / "bronze" / SOURCE_NAME
LOG_DIR = PROJECT_ROOT / "logs"
METADATA_DIR = PROJECT_ROOT / "metadata"

RAW_DIR.mkdir(parents=True, exist_ok=True)
BRONZE_DIR.mkdir(parents=True, exist_ok=True)
LOG_DIR.mkdir(parents=True, exist_ok=True)
METADATA_DIR.mkdir(parents=True, exist_ok=True)

LOG_FILE = LOG_DIR / f"{SOURCE_NAME}_ingestion_log.jsonl"
MANIFEST_FILE = METADATA_DIR / f"{SOURCE_NAME}_manifest_{BA
```

Output Cell 1

```
c:\Users\barath\AppData\Local\anaconda3\Lib\site-
packages\pandas\core\computation\expressions.py:22: UserWarning: Pandas requires version
'2.10.2' or newer of 'numexpr' (version '2.10.1' currently installed).
  from pandas.core.computation.check import NUMEXPR_INSTALLED


DummyJSON ingestion completed successfully
```

```
Source mode: api
Raw dir: C:\Users\barath\recomart-pipeline\data\raw\dummyjson\load_date=2026-04-29\load_hour=06
Bronze dir: C:\Users\barath\recomart-pipeline\data\bronze\dummyjson
Manifest: C:\Users\barath\recomart-pipeline\metadata\dummyjson_manifest_20260429T064819Z.json

Summary:
      dataset  rows  columns  \
0    products   194       32
1  categories    24        8

                                        output_path source_mode
0  C:\Users\barath\recomart-pipeline\data\bronze\...         api
1  C:\Users\barath\recomart-pipeline\data\bronze\...         api
```

Code Cell 2

```
No code found.
```

Output Cell 2

```
No saved output found in notebook.
```

Notebook: retailrocket_ingestion_executed.ipynb

Path:
runs\prefect\20260429_121758\executed_notebooks\ingest_retailrocket\retailrocket_ingestion_executed.ipynb

Code Cell 1

```
# DM4ML -Assignment - Ingestion - File

# ============================================================
# Retailrocket Ingestion Notebook / Script
# Purpose:
#   - Download Retailrocket dataset via kagglehub
#   - Copy raw files into partitioned local data lake folders
#   - Log success/failure events
#   - Create a manifest for audit/lineage
#   - Build bronze parquet datasets for downstream use
# ============================================================

# -----------------------------
# 0) Optional: install packages
# Uncomment if needed in Jupyter
# -----------------------------
# import sys
# !{sys.executable} -m pip install kagglehub pandas pyarrow

import json
import shutil
import time
from pathlib import Path
from datetime import datetime, timezone

import pandas as pd
import kagglehub

# ============================
# 1) CONFIG
# ============================
PROJECT_ROOT = Path.cwd().resolve()
while not (PROJECT_ROOT / "src").exists() and PROJECT_ROOT != PROJECT_ROOT.parent:
    PROJECT_ROOT = PROJECT_ROOT.parent
DATA_ROOT = PROJECT_ROOT / "data"
SOURCE_NAME = "retailrocket"
DATASET_ID = "retailrocket/ecommerce-dataset"

UTC_NOW = datetime.now(timezone.utc)
LOAD_DATE = UTC_NOW.strftime("%Y-%m-%d")
LOAD_HOUR = UTC_NOW.strftime("%H")
BATCH_ID = UTC_NOW.strftime("%Y%m%dT%H%M%SZ")
INGESTION_TS = UTC_NOW.isoformat()

RAW_DIR = PROJECT_ROOT / "data" / "raw" / SOURCE_NAME / f"load_date={LOAD_DATE}" /
f"load_hour={LOAD_HOUR}"
BRONZE_DIR = PROJECT_ROOT / "data" / "bronze" / SOURCE_NAME
LOG_DIR = PROJECT_ROOT / "logs"
METADATA_DIR = PROJECT_ROOT / "metadata"

RAW_DIR.mkdir(parents=True, exist_ok=True)
BRONZE_DIR.mkdir(parents=True, exist_ok=True)
LOG_DIR.mkdir(parents=True, exist_ok=True)
METADATA_DIR.mkdir(parents=True, exist_ok=True)

LOG_FILE = LOG_DIR / f"{SOURCE
```

Output Cell 1

```
c:\Users\barath\AppData\Local\anaconda3\Lib\site-
packages\pandas\core\computation\expressions.py:22: UserWarning: Pandas requires version
'2.10.2' or newer of 'numexpr' (version '2.10.1' currently installed).
  from pandas.core.computation.check import NUMEXPR_INSTALLED


Downloaded dataset to: C:\Users\barath\.cache\kagglehub\datasets\retailrocket\ecommerce-
dataset\versions\2


Raw files copied to: C:\Users\barath\recomart-
pipeline\data\raw\retailrocket\load_date=2026-04-29\load_hour=06
Manifest saved to: C:\Users\barath\recomart-
pipeline\metadata\retailrocket_manifest_20260429T064820Z.json
```

Code Cell 2

```
No code found.
```

Output Cell 2

```
No saved output found in notebook.
```

Notebook: run_validation_executed.ipynb

Path: runs\prefect\20260429_121758\executed_notebooks\validate_raw\run_validation_executed.ipynb

Code Cell 1

```
# DM4ML - Assignment - Validation + Auto-Fix + Revalidation + PDF Report

import json
from pathlib import Path
from datetime import datetime, timezone

import pandas as pd

from reportlab.lib import colors
from reportlab.lib.pagesizes import A4
from reportlab.lib.styles import getSampleStyleSheet
from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer, Table, TableStyle,
PageBreak

# ============================================================
# RecoMart Validation - robust file discovery + validation
# with remediation, revalidation, and PDF reporting
# ============================================================

PROJECT_ROOT = Path.cwd().resolve()
while not (PROJECT_ROOT / "src").exists() and PROJECT_ROOT != PROJECT_ROOT.parent:
    PROJECT_ROOT = PROJECT_ROOT.parent

RAW_ROOT = PROJECT_ROOT / "data" / "raw"
BRONZE_ROOT = PROJECT_ROOT / "data" / "bronze"

REMEDIATED_RAW_ROOT = PROJECT_ROOT / "data" / "remediated" / "raw"
REMEDIATED_BRONZE_ROOT = PROJECT_ROOT / "data" / "remediated" / "bronze"

REPORT_ROOT = PROJECT_ROOT / "reports" / "validation"
LOG_ROOT = PROJECT_ROOT / "logs"

for folder in [REPORT_ROOT, LOG_ROOT, REMEDIATED_RAW_ROOT, REMEDIATED_BRONZE_ROOT]:
    folder.mkdir(parents=True, exist_ok=True)

UTC_NOW = datetime.now(timezone.utc)
RUN_ID = UTC_NOW.strftime("%Y%m%dT%H%M%SZ")
RUN_TS = UTC_NOW.isoformat()

LOG_FILE = LOG_ROOT / f"validation_log_{RUN_ID}.jsonl"
INITIAL_ISSUES_FILE = REPORT_ROOT / f"validation_issues_initial_{RUN_ID}.csv"
INITIAL_SUMMARY_FILE = REPORT_ROOT / f"validation_summary_initial_{RUN_ID}.csv"
FINAL_ISSUES_FILE = REPORT_ROOT / f"validation_issues_revalidated_{RUN_ID}.csv"
FINAL_SUMMARY_FILE = REPORT_ROOT / f"validation_summary_revalidated_{RUN_ID}.csv"
FIX_LOG_FILE = REPORT_ROOT / f"fix_log_{RUN_ID}.csv"
REPORT_FILE = REPO
```

Output Cell 1

```
c:\Users\barath\AppData\Local\anaconda3\Lib\site-
packages\pandas\core\computation\expressions.py:22: UserWarning: Pandas requires version
'2.10.2' or newer of 'numexpr' (version '2.10.1' currently installed).
  from pandas.core.computation.check import NUMEXPR_INSTALLED


PROJECT_ROOT: C:\Users\barath\recomart-pipeline
RAW_ROOT: C:\Users\barath\recomart-pipeline\data\raw
BRONZE_ROOT: C:\Users\barath\recomart-pipeline\data\bronze
Discovered files:
```

```
- events_csv: C:\Users\barath\recomart-
pipeline\data\raw\retailrocket\load_date=2026-04-29\load_hour=06\events.csv
- category_tree_csv: C:\Users\barath\recomart-
pipeline\data\raw\retailrocket\load_date=2026-04-26\load_hour=19\category_tree.csv
- item_properties_part1_csv: C:\Users\barath\recomart-
pipeline\data\raw\retailrocket\load_date=2026-04-29\load_hour=06\item_properties_part1.csv
- item_properties_part2_csv: C:\Users\barath\recomart-
pipeline\data\raw\retailrocket\load_date=2026-04-29\load_hour=06\item_properties_part2.csv
- products_raw_json: C:\Users\barath\recomart-
pipeline\data\raw\dummyjson\load_date=2026-04-29\load_hour=06\products_raw.json
- categories_raw_json: C:\Users\barath\recomart-pipeline\data\raw\dummyjson\load_date=20
```

Code Cell 2

```
No code found.
```

Output Cell 2

```
No saved output found in notebook.
```

5. Logging and Audit Trail Evidence

The latest execution evidence indicates an initial status of **FAIL\n",** and a final status of **PASS\n",**. The following artifacts provide supporting evidence for ingestion, validation, remediation, and reporting.

Log / Report File	Relative Path	Last Modified	Size
run_validation.txt	src\02-validation\run_validation.txt	2026-04-28 20:14:12	46.94 KB
data_quality_report_20260429T111342Z.json	reports\validation\data_quality_report_20260429T111342Z.json	2026-04-29 16:47:43	6.66 KB
data_quality_report_20260429T111342Z.pdf	reports\validation\data_quality_report_20260429T111342Z.pdf	2026-04-29 16:47:44	8.49 KB
fix_log_20260429T111342Z.csv	reports\validation\fix_log_20260429T111342Z.csv	2026-04-29 16:46:38	3.43 KB
validation_summary_revalidated_20260429T111342Z.csv	reports\validation\validation_summary_revalidated_20260429T111342Z.csv	2026-04-29 16:47:43	0.41 KB
validation_issues_revalidated_20260429T111342Z.csv	reports\validation\validation_issues_revalidated_20260429T111342Z.csv	2026-04-29 16:47:43	0.51 KB

6. Raw and Bronze Storage Structure

A structured folder layout is used to store ingested data. Raw files are organized under source-specific directories, while bronze data contains normalized parquet outputs for downstream processing.

Raw Data Folder Tree

```
raw/
■■■ dummyjson/
■   ■■■ load_date=2026-04-29/
■       ■■■ load_hour=11/
■           ■■■ categories_raw.json
■           ■■■ products_raw.json
■■■ retailrocket/
    ■■■ load_date=2026-04-29/
        ■■■ load_hour=11/
            ■■■ category_tree.csv
            ■■■ events.csv
            ■■■ item_properties_part1.csv
            ■■■ item_properties_part2.csv
```

Bronze Data Folder Tree

```
bronze/
■■■ dummyjson/
■   ■■■ categories.parquet
■   ■■■ ingestion_summary.csv
■   ■■■ products.parquet
■■■ retailrocket/
    ■■■ category_tree.parquet
    ■■■ events.parquet
    ■■■ ingestion_summary.csv
```

■■■ item_properties.parquet

7. Validation / Execution Log Preview

The preview below captures the beginning of the latest validation or execution text file discovered in the project. This provides traceable evidence of discovered source files, execution statuses, and generated outputs.


```
{
 "cells": [
  {
   "cell_type": "code",
   "execution_count": 2,
   "id": "fef32579-f890-4b53-8d24-2fcae64a4914",
   "metadata": {},
   "outputs": [
    {
     "name": "stdout",
     "output_type": "stream",
     "text": [
      "PROJECT_ROOT: C:\\Users\\barath\\recomart-pipeline\n",
      "RAW_ROOT: C:\\Users\\barath\\recomart-pipeline\\data\\raw\n",
      "BRONZE_ROOT: C:\\Users\\barath\\recomart-pipeline\\data\\bronze\n",
      "\n",
      "Discovered files:\n",
      "- events_csv: C:\\Users\\barath\\recomart-
pipeline\\data\\raw\\retailrocket\\load_date=2026-04-28\\load_hour=14\\events.csv\n",
      "- category_tree_csv: C:\\Users\\barath\\recomart-
pipeline\\data\\raw\\retailrocket\\load_date=2026-04-26\\load_hour=19\\category_tree.csv\n",
      "- item_properties_part1_csv: C:\\Users\\barath\\recomart-pipeline\\data\\raw\\retailrock
et\\load_date=2026-04-28\\load_hour=14\\item_properties_part1.csv\n",
      "- item_properties_part2_csv: C:\\Users\\barath\\recomart-pipeline\\data\\raw\\retailrock
et\\load_date=2026-04-28\\load_hour=14\\item_properties_part2.csv\n",
      "- products_raw_json: C:\\Users\\barath\\recomart-
pipeline\\data\\raw\\dummyjson\\load_date=2026-04-28\\load_hour=14\\products_raw.json\n",
      "- categories_raw_json: C:\\Users\\barath\\recomart-
pipeline\\data\\raw\\dummyjson\\load_date=2026-04-28\\load_hour=14\\categories_raw.json\n",
      "- products_parquet: C:\\Users\\barath\\recomart-
pipeline\\data\\bronze\\dummyjson\\products.parquet\n",
      "- categories_parquet: C:\\Users\\barath\\recomart-
pipeline\\data\\bronze\\dummyjson\\categories.parquet\n",
      "\n",
      "Initial status: FAIL\n",
      "Final status: PASS\n",
      "Initial issues file: C:\\Users\\barath\\recomart-
pipeline\\reports\\validation\\validation_issues_initial_20260428T144031Z.csv\n",
      "Initial summary file: C:\\Users\\barath\\recomart-
pipeline\\reports\\validation\\validation_summary_initial_20260428T144031Z.csv\n",
      "Revalidation issues file: C:\\Users\\barath\\recomart-
pipeline\\reports\\validation\\validation_issues_revalidated_20260428T144031Z.csv\n",
      "Revalidation summary file: C:\\Users\\barath\\recomart-
pipeline\\reports\\validation\\validation_summary_revalidated_20260428T144031Z.csv\n",
      "Fix log file: C:\\Users\\barath\\recomart-
pipeline\\reports\\validation\\fix_log_20260428T144031Z.csv\n",
      "JSON report file: C:\\Users\\barath\\recomart-
pipeline\\reports\\validation\\data_quality_report_20260428T144031Z.json\n",
      "PDF report file: C:\\Users\\barath\\recomart-
pipeline\\reports\\validation\\data_quality_report_20260428T144031Z.pdf\n"
```


8. JSON Report Preview

If a JSON report exists, the preview below captures the first lines for quick inspection of generated reporting metadata.


```
{
  "run_id": "20260429T111342Z",
  "run_ts": "2026-04-29T11:13:42.777701+00:00",
  "project_root": "C:\\Users\\barath\\recomart-pipeline",
  "initial_validation": {
    "run_id": "20260429T111342Z",
    "run_ts": "2026-04-29T11:13:42.777701+00:00",
    "phase": "initial",
    "project_root": "C:\\Users\\barath\\recomart-pipeline",
    "raw_root": "C:\\Users\\barath\\recomart-pipeline\\data\\raw",
    "bronze_root": "C:\\Users\\barath\\recomart-pipeline\\data\\bronze",
    "discovered_files": {
      "events_csv": "C:\\Users\\barath\\recomart-
pipeline\\data\\raw\\retailrocket\\load_date=2026-04-29\\load_hour=11\\events.csv",
      "category_tree_csv": "C:\\Users\\barath\\recomart-
pipeline\\data\\raw\\retailrocket\\load_date=2026-04-29\\load_hour=11\\category_tree.csv",
      "item_properties_part1_csv": "C:\\Users\\barath\\recomart-pipeline\\data\\raw\\retailrock
et\\load_date=2026-04-29\\load_hour=11\\item_properties_part1.csv",
      "item_properties_part2_csv": "C:\\Users\\barath\\recomart-pipeline\\data\\raw\\retailrock
et\\load_date=2026-04-29\\load_hour=11\\item_properties_part2.csv",
      "products_raw_json": "C:\\Users\\barath\\recomart-
pipeline\\data\\raw\\dummyjson\\load_date=2026-04-29\\load_hour=11\\products_raw.json",
```

```
      "categories_raw_json": "C:\\Users\\barath\\recomart-
pipeline\\data\\raw\\dummyjson\\load_date=2026-04-29\\load_hour=11\\categories_raw.json",
      "products_parquet": "C:\\Users\\barath\\recomart-
pipeline\\data\\bronze\\dummyjson\\products.parquet",
      "categories_parquet": "C:\\Users\\barath\\recomart-
pipeline\\data\\bronze\\dummyjson\\categories.parquet"
    },
    "overall_status": "FAIL",
    "issues_file": "C:\\Users\\barath\\recomart-
pipeline\\reports\\validation\\validation_issues_initial_20260429T111342Z.csv",
    "summary_file": "C:\\Users\\barath\\recomart-
pipeline\\reports\\validation\\validation_summary_initial_20260429T111342Z.csv",
    "datasets": [
      {
        "phase": "initial",
        "dataset": "retailrocket_events",
        "rows": 2756101,
        "columns": 5,
        "errors": 0,
        "warnings": 2734104,
        "status": "PASS"
      },
      {
```

9. Conclusion

Based on the discovered project structure, the pipeline demonstrates multi-source data ingestion, structured raw and bronze storage, and generation of logs and reports that support monitoring and auditability. This PDF was generated directly from the available project files to create submission-ready documentation for the Data Collection and Ingestion stage.